

Python Programming — Comprehensive Reference

Python is a high-level, interpreted, general-purpose programming language. Created by Guido van Rossum and first released in 1991, Python's design philosophy emphasizes code readability and simplicity. It supports multiple programming paradigms including procedural, object-oriented, and functional programming.

Python is widely used in web development, data science, artificial intelligence, machine learning, automation, scientific computing, and more. Its extensive standard library and vibrant ecosystem of third-party packages make it one of the most popular languages in the world.

1. Variables and Data Types

Python is dynamically typed, meaning you do not need to declare variable types explicitly. The interpreter infers the type at runtime based on the assigned value.

```
# Integer
age = 25
year = 2026

# Float
pi = 3.14159
temperature = 36.6

# String
name = "Abhishek"
greeting = 'Hello, World!'

# Boolean
is_active = True
is_deleted = False

# NoneType
result = None

# Check type
print(type(age))      # <class 'int'>
print(type(pi))       # <class 'float'>
print(type(name))     # <class 'str'>
print(type(is_active)) # <class 'bool'>
```

2. String Operations

Strings in Python are immutable sequences of Unicode characters. Python provides a rich set of built-in methods for string manipulation.

```
s = "Hello, Python!"

# Basic operations
print(len(s))          # 14
print(s.upper())        # HELLO, PYTHON!
print(s.lower())        # hello, python!
print(s.strip())        # removes whitespace
print(s.replace("Python", "World")) # Hello, World!
print(s.split(", "))    # ['Hello', 'Python!']

# Slicing
print(s[0:5])           # Hello
print(s[-7:])            # Python!
print(s[::-1])           # !nohtyP ,olleH

# f-strings (Python 3.6+)
name = "Abhishek"
score = 97
print(f"Name: {name}, Score: {score}/150")
```

Lists, Tuples, and Sets

3. Lists

Lists are ordered, mutable collections. They can hold elements of different data types and support indexing, slicing, and a wide range of methods.

```
# Creating lists
fruits = ["apple", "banana", "cherry"]
numbers = [1, 2, 3, 4, 5]
mixed = [1, "hello", 3.14, True]

# Accessing elements
print(fruits[0])      # apple
print(fruits[-1])     # cherry

# Modifying
fruits.append("mango")
fruits.insert(1, "orange")
fruits.remove("banana")
popped = fruits.pop()

# List comprehension
squares = [x**2 for x in range(10)]
evens = [x for x in range(20) if x % 2 == 0]

# Useful methods
numbers.sort()
numbers.reverse()
print(len(numbers))   # 5
print(sum(numbers))   # 15
print(min(numbers))   # 1
print(max(numbers))   # 5
```

4. Tuples

Tuples are ordered, immutable collections. Once created, their elements cannot be changed. They are faster than lists and commonly used for fixed data.

```
# Creating tuples
point = (3, 4)
rgb = (255, 128, 0)
single = (42,)      # Note the trailing comma

# Unpacking
x, y = point
```

```
r, g, b = rgb

# Named tuple
from collections import namedtuple
Person = namedtuple('Person', ['name', 'age', 'city'])
p = Person("Abhishek", 21, "Bengaluru")
print(p.name)          # Abhishek
print(p.age)           # 21
```

5. Sets

Sets are unordered collections of unique elements. They are ideal for membership testing and eliminating duplicate entries.

```
# Creating sets
primes = {2, 3, 5, 7, 11}
vowels = set("aeiou")

# Operations
primes.add(13)
primes.discard(2)

a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
print(a | b)    # Union:      {1, 2, 3, 4, 5, 6}
print(a & b)    # Intersection: {3, 4}
print(a - b)    # Difference:  {1, 2}
print(a ^ b)    # Symmetric:   {1, 2, 5, 6}
```

Dictionaries and Control Flow

6. Dictionaries

Dictionaries are ordered (Python 3.7+), mutable key-value stores. Keys must be hashable (strings, numbers, tuples). They are one of the most used data structures in Python.

```
# Creating dictionaries
student = {
    "name": "Abhishek",
    "age": 21,
    "skills": ["Python", "FastAPI", "React"],
    "cgpa": 8.3
}

# Accessing
print(student["name"])          # Abhishek
print(student.get("gpa", "N/A")) # N/A (default)

# Modifying
student["city"] = "Bengaluru"
student["age"] = 22
del student["cgpa"]

# Iterating
for key in student:
    print(key)

for key, value in student.items():
    print(f"{key}: {value}")

# Dictionary comprehension
squares = {x: x**2 for x in range(1, 6)}
# {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# Merging (Python 3.9+)
defaults = {"theme": "dark", "lang": "en"}
overrides = {"lang": "hi", "timezone": "IST"}
config = defaults | overrides
```

7. Control Flow — Conditionals

Python uses indentation (typically 4 spaces) to define code blocks. The if-elif-else construct handles conditional logic.

```
score = 85
```

```

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"

print(f"Grade: {grade}") # Grade: B

# Ternary operator
status = "pass" if score >= 50 else "fail"

# Match statement (Python 3.10+)
command = "quit"
match command:
    case "quit":
        print("Exiting...")
    case "help":
        print("Showing help...")
    case _:
        print("Unknown command")

```

8. Loops

```

# for loop
for i in range(5):
    print(i)                # 0 1 2 3 4

# while loop
count = 0
while count < 5:
    count += 1

# Loop control
for i in range(10):
    if i == 3: continue     # skip 3
    if i == 7: break        # stop at 7
    print(i)

# enumerate and zip
names = ["Alice", "Bob", "Charlie"]
for i, name in enumerate(names, start=1):
    print(f"{i}. {name}")

scores = [85, 92, 78]

```

```
for name, score in zip(names, scores):  
    print(f"{name}: {score}")
```

Functions and Functional Programming

9. Defining Functions

Functions in Python are first-class objects — they can be assigned to variables, passed as arguments, and returned from other functions. Use the `def` keyword to define them.

```
def greet(name, greeting="Hello"):
    """Return a greeting message."""
    return f"{greeting}, {name}!"

print(greet("Abhishek"))          # Hello, Abhishek!
print(greet("Abhishek", "Hi"))    # Hi, Abhishek!

# *args and **kwargs
def summarize(*args, **kwargs):
    print("Positional:", args)
    print("Keyword:", kwargs)

summarize(1, 2, 3, name="Abhishek", city="Bengaluru")

# Type hints (Python 3.5+)
def add(a: int, b: int) -> int:
    return a + b

# Docstrings
def compute_area(radius: float) -> float:
    """
    Compute the area of a circle.

    Args:
        radius: The radius of the circle.

    Returns:
        The area as a float.
    """
    import math
    return math.pi * radius ** 2
```

10. Lambda Functions

```
# Lambda – anonymous single-expression functions
square = lambda x: x ** 2
add = lambda x, y: x + y

# Common use with built-ins
```



```
numbers = [5, 2, 8, 1, 9, 3]
sorted_nums = sorted(numbers, key=lambda x: -x)

students = [{"name": "A", "gpa": 3.8}, {"name": "B", "gpa": 3.5}]
top = sorted(students, key=lambda s: s["gpa"], reverse=True)
```

11. Map, Filter, Reduce

```
from functools import reduce

nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# map – apply function to each element
squares = list(map(lambda x: x**2, nums))

# filter – keep elements matching condition
evens = list(filter(lambda x: x % 2 == 0, nums))

# reduce – accumulate to single value
total = reduce(lambda acc, x: acc + x, nums)
print(total) # 55

# Generators – memory efficient iteration
def fibonacci(n):
    a, b = 0, 1
    for _ in range(n):
        yield a
        a, b = b, a + b

for fib in fibonacci(10):
    print(fib, end=" ") # 0 1 1 2 3 5 8 13 21 34
```

Object-Oriented Programming

12. Classes and Objects

Python supports full object-oriented programming with classes, inheritance, encapsulation, and polymorphism. Everything in Python is an object.

```
class BankAccount:
    interest_rate = 0.05      # class variable

    def __init__(self, owner: str, balance: float = 0):
        self.owner = owner    # instance variable
        self._balance = balance

    @property
    def balance(self):
        return self._balance

    def deposit(self, amount: float):
        if amount <= 0:
            raise ValueError("Amount must be positive")
        self._balance += amount
        return self

    def withdraw(self, amount: float):
        if amount > self._balance:
            raise ValueError("Insufficient funds")
        self._balance -= amount
        return self

    def __repr__(self):
        return f"BankAccount(owner={self.owner!r}, balance={self._balance})"

    def __str__(self):
        return f"{self.owner}'s account: Rs.{self._balance:,.2f}"

# Usage
acc = BankAccount("Abhishek", 10000)
acc.deposit(5000).withdraw(2000) # method chaining
print(acc) # Abhishek's account: Rs.13,000.00
```

13. Inheritance

```
class SavingsAccount(BankAccount):
    def __init__(self, owner, balance=0, min_balance=1000):
        super().__init__(owner, balance)
```

```

        self.min_balance = min_balance

    def withdraw(self, amount):
        if self._balance - amount < self.min_balance:
            raise ValueError("Cannot go below minimum balance")
        return super().withdraw(amount)

class CurrentAccount(BankAccount):
    def __init__(self, owner, balance=0, overdraft=5000):
        super().__init__(owner, balance)
        self.overdraft = overdraft

    def withdraw(self, amount):
        if amount > self._balance + self.overdraft:
            raise ValueError("Overdraft limit exceeded")
        self._balance -= amount
        return self

# Polymorphism
accounts = [SavingsAccount("Alice", 5000), CurrentAccount("Bob", 2000)]
for acc in accounts:
    print(type(acc).__name__, acc.balance)

```

Error Handling and File I/O

14. Exceptions

Python uses try-except blocks to handle exceptions gracefully. You can catch specific exceptions, define cleanup actions with finally, and raise custom exceptions.

```
# Basic exception handling
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(f"Error: {e}")
except (TypeError, ValueError) as e:
    print(f"Type/Value error: {e}")
else:
    print("No exception occurred")
finally:
    print("This always runs")

# Custom exceptions
class InsufficientFundsError(Exception):
    def __init__(self, amount, balance):
        self.amount = amount
        self.balance = balance
        super().__init__(
            f"Cannot withdraw {amount}. Balance: {balance}"
        )

# Context managers
class DatabaseConnection:
    def __enter__(self):
        print("Connecting to DB...")
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        print("Closing connection...")
        return False # Don't suppress exceptions

with DatabaseConnection() as db:
    print("Running queries...")
```

15. File I/O

```
# Reading files
with open("data.txt", "r", encoding="utf-8") as f:
    content = f.read() # entire file
```

```

        lines = f.readlines()          # list of lines

# Writing files
with open("output.txt", "w", encoding="utf-8") as f:
    f.write("Hello, World!\n")
    f.writelines(["Line 1\n", "Line 2\n"])

# Appending
with open("log.txt", "a") as f:
    f.write("New log entry\n")

# Working with JSON
import json

data = {"name": "Abhishek", "skills": ["Python", "FastAPI"]}
with open("data.json", "w") as f:
    json.dump(data, f, indent=2)

with open("data.json", "r") as f:
    loaded = json.load(f)

# CSV handling
import csv
with open("students.csv", "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=["name", "score"])
    writer.writeheader()
    writer.writerow({"name": "Abhishek", "score": 97})

```

Modules, Packages, and the Standard Library

16. Importing Modules

Python's module system allows you to organise code into reusable files. The standard library ships with hundreds of modules covering everything from math to networking.

```
# Basic imports
import math
import os
import sys
from datetime import datetime, timedelta
from collections import defaultdict, Counter, deque
from itertools import chain, combinations, permutations

# Math
print(math.sqrt(144))          # 12.0
print(math.ceil(4.2))          # 5
print(math.floor(4.8))         # 4
print(math.log(100, 10))       # 2.0

# datetime
now = datetime.now()
print(now.strftime("%Y-%m-%d %H:%M"))
tomorrow = now + timedelta(days=1)
diff = tomorrow - now

# OS operations
cwd = os.getcwd()
files = os.listdir(".")
os.makedirs("new_dir", exist_ok=True)
path = os.path.join("folder", "file.txt")
exists = os.path.exists(path)
```

17. Collections Module

```
from collections import defaultdict, Counter, deque

# defaultdict – no KeyError
word_count = defaultdict(int)
for word in "the quick brown fox jumps over the lazy dog".split():
    word_count[word] += 1

# Counter
scores = [85, 92, 85, 78, 92, 92, 85]
c = Counter(scores)
```

```

print(c.most_common(2))      # [(92, 3), (85, 3)]

# deque – efficient append/pop from both ends
q = deque([1, 2, 3])
q.appendleft(0)              # [0, 1, 2, 3]
q.append(4)                  # [0, 1, 2, 3, 4]
q.popleft()                  # returns 0
q.rotate(1)                  # [4, 1, 2, 3]

```

18. itertools Module

```

from itertools import combinations, permutations, product, groupby

items = [1, 2, 3, 4]

# All 2-element combinations
for combo in combinations(items, 2):
    print(combo)    # (1,2), (1,3), (1,4), (2,3)...

# All permutations
perms = list(permutations([1, 2, 3])) # 6 permutations

# Cartesian product
for pair in product("AB", "12"):
    print(pair)    # (A,1), (A,2), (B,1), (B,2)

# groupby
data = [("a", 1), ("a", 2), ("b", 3), ("b", 4)]
for key, group in groupby(data, key=lambda x: x[0]):
    print(key, list(group))

```

Decorators, Generators, and Advanced Patterns

19. Decorators

Decorators are a powerful Python feature that allows you to modify the behaviour of functions or classes. They are widely used in web frameworks, testing, and caching.

```
import time
import functools

# Basic decorator
def timer(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        end = time.perf_counter()
        print(f"[{func.__name__}] took {end - start:.4f}s")
        return result
    return wrapper

# Decorator with arguments
def retry(max_attempts=3, delay=1.0):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(max_attempts):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    if attempt == max_attempts - 1:
                        raise
                    time.sleep(delay)
            return wrapper
        return decorator

@timer
@retry(max_attempts=3, delay=0.5)
def fetch_data(url: str):
    # simulate API call
    return {"data": "result"}

# Class decorator
def singleton(cls):
    instances = {}
    @functools.wraps(cls)
    def get_instance(*args, **kwargs):
```



```

        if cls not in instances:
            instances[cls] = cls(*args, **kwargs)
        return instances[cls]
    return get_instance

@singleton
class Config:
    def __init__(self):
        self.debug = False

```

20. Generators and Coroutines

```

# Generator function
def chunked(lst, size):
    """Yield successive chunks from lst."""
    for i in range(0, len(lst), size):
        yield lst[i:i + size]

data = list(range(100))
for chunk in chunked(data, 10):
    print(chunk)

# Generator expression (memory efficient)
total = sum(x**2 for x in range(1_000_000))

# Infinite generator
def counter(start=0, step=1):
    n = start
    while True:
        yield n
        n += step

from itertools import islice
first_10 = list(islice(counter(0, 2), 10))
# [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

```

Async Programming and Type System

21. Async / Await

Python's asyncio library enables asynchronous programming, allowing you to write concurrent code using coroutines. This is essential for I/O-bound tasks like web requests, database queries, and file operations.

```
import asyncio
import aiohttp

# Basic coroutine
async def say_hello(name: str, delay: float):
    await asyncio.sleep(delay)
    print(f"Hello, {name}!")

# Running coroutines concurrently
async def main():
    # Sequential – slow
    await say_hello("Alice", 1)
    await say_hello("Bob", 1)

    # Concurrent – fast
    await asyncio.gather(
        say_hello("Alice", 1),
        say_hello("Bob", 1),
        say_hello("Charlie", 1)
    )

asyncio.run(main())

# Async HTTP requests
async def fetch(session, url):
    async with session.get(url) as response:
        return await response.json()

async def fetch_all(urls):
    async with aiohttp.ClientSession() as session:
        tasks = [fetch(session, url) for url in urls]
        return await asyncio.gather(*tasks)

# Async context manager
class AsyncDB:
    async def __aenter__(self):
        await asyncio.sleep(0.1) # simulate connect
        return self

    async def __aexit__(self, *args):
```

```

        await asyncio.sleep(0.1) # simulate disconnect

async def use_db():
    async with AsyncDB() as db:
        pass # use db

```

22. Type Hints and Dataclasses

```

from typing import Optional, List, Dict, Union, Any, Callable
from dataclasses import dataclass, field

```

```

# Type hints
def process(
    items: List[str],
    config: Optional[Dict[str, Any]] = None,
    callback: Callable[[str], bool] = None
) -> Union[List[str], None]:
    return items

# Dataclasses – auto-generate __init__, __repr__, __eq__
@dataclass
class Student:
    name: str
    age: int
    skills: List[str] = field(default_factory=list)
    cgpa: float = 0.0
    _id: int = field(default=0, repr=False)

    def add_skill(self, skill: str) -> None:
        self.skills.append(skill)

    @property
    def is_distinction(self) -> bool:
        return self.cgpa >= 8.5

s = Student("Abhishek", 21, ["Python", "FastAPI"], 8.3)
print(s)
print(s.is_distinction) # False

```

Testing, Packaging, and Best Practices

23. Unit Testing with pytest

Testing is a fundamental part of software engineering. Python's pytest framework makes it easy to write and run tests, with powerful features like fixtures, parametrize, and mocking.

```
# test_bank.py
import pytest
from bank import BankAccount, InsufficientFundsError

@pytest.fixture
def account():
    return BankAccount("Abhishek", balance=10000)

def test_initial_balance(account):
    assert account.balance == 10000

def test_deposit(account):
    account.deposit(5000)
    assert account.balance == 15000

def test_deposit_negative_raises(account):
    with pytest.raises(ValueError, match="must be positive"):
        account.deposit(-100)

def test_withdraw(account):
    account.withdraw(3000)
    assert account.balance == 7000

@pytest.mark.parametrize("amount, expected", [
    (1000, 9000),
    (5000, 5000),
    (10000, 0),
])
def test_withdraw_parametrized(account, amount, expected):
    account.withdraw(amount)
    assert account.balance == expected

# Run: pytest test_bank.py -v
```

24. Virtual Environments and Project Structure

```
# Create and activate venv
python -m venv venv
source venv/bin/activate      # Linux/Mac
```

```

venv\Scripts\activate          # Windows

# Install and freeze dependencies
pip install fastapi uvicorn
pip freeze > requirements.txt
pip install -r requirements.txt

# Recommended project structure
my_project/
  src/
    my_package/
      __init__.py
      core.py
      utils.py
  tests/
    test_core.py
    test_utils.py
  docs/
  .env
  .gitignore
  requirements.txt
  README.md
  pyproject.toml

```

25. Python Best Practices

Following PEP 8 style guidelines, writing docstrings, using type hints, and keeping functions small and focused are the hallmarks of professional Python code.

```

# PEP 8 – Python style guide highlights
# - 4 spaces indentation (no tabs)
# - max 79 characters per line
# - two blank lines between top-level definitions
# - one blank line between methods
# - snake_case for variables/functions
# - PascalCase for classes
# - UPPER_CASE for constants

MAX_RETRIES = 3
DEFAULT_TIMEOUT = 30.0

def calculate_compound_interest(
    principal: float,
    rate: float,
    periods: int,
    compounds_per_year: int = 12
) -> float:

```

```

"""
Calculate compound interest.

Args:
    principal: Initial investment amount.
    rate: Annual interest rate (0.05 = 5%).
    periods: Number of years.
    compounds_per_year: Compounding frequency.

Returns:
    Final amount after compound interest.

Example:
    >>> calculate_compound_interest(10000, 0.05, 10)
    16470.09
"""
return principal * (1 + rate / compounds_per_year) ** (compounds_per_year * periods)

```

This document covers the core pillars of Python programming — from basic data types through to async programming and testing. Mastery of these concepts forms the foundation for building production-grade systems in Python.